

open the `.config` file, and there will be an entry as shown below:

```
CONFIG_MY_DRIVER=m
```

Here, the driver is configured to be built as a module. Also, one thing worth noting is that the symbol 'MY_DRIVER' in Kconfig is prefixed with `CONFIG_`.

Now, just adding an entry in the Kconfig file and configuration alone won't compile the driver. There has to be the corresponding change in makefile as well. So, add the following line to makefile:

```
obj-$(CONFIG_MYDRIVER) += my_driver.o
```

After the kernel is compiled, the module `my_driver.ko` will be placed at `drivers/char/`. This module can be inserted in the kernel with the following command:

```
$ insmod my_driver.ko
```

Aren't these configuration symbols needed in the C code? Yes, or else how will the conditional compilation be taken care of? How are these symbols included in C code? During the kernel compilation, the Kconfig and `.config` files are read, and are used to generate the C header file named `autoconf.h`. This is placed at `include/generated` and contains the `#defines` for the configuration symbols. These symbols are used by the C code to conditionally compile the required code.

Now, let's suppose I have configured the kernel and that it works fine with this configuration. And, if I make some new changes in the kernel configuration, the earlier ones will be overwritten. In order to avoid this from happening, we can save `.config` file in the `arch/arm/configs` directory with a name like `my_config`, for instance. And next time, we can execute the following command to configure the kernel with older options:

```
$ make my_config_defconfig
```

Linux Support Packages (LSP)/Board Support Packages (BSP)

One of the most important and probably the most challenging thing in porting is the development of Board Support Packages (BSP). BSP development is a one-time effort during the product development lifecycle and, obviously, the most critical. As we have discussed, porting involves architecture porting and board porting. Board porting involves board-specific initialisation code that includes initialisation of the various interfaces such as memory, peripherals such as serial, and i2c, which in turn, involves the driver porting.

There are two categories of drivers. One is the standard device driver such as the i2c driver and block driver located at the standard directory location. Another is the custom interface or device driver, which includes the

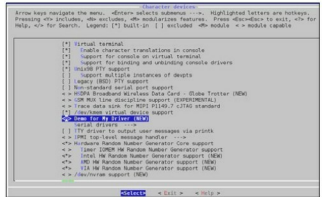


Figure 3: Menu option for *My Driver*

board-specific custom code and needs to be specifically brought in with the kernel. And this collection of board-specific initialisation and custom code is referred to as a Board Support Package or, in Linux terminology, a LSP. In simple words, whatever software code you require (which is specific to the target platform) to boot up the target with the operating system can be called LSP.

Components of LSP

As the name itself suggests, BSP is dependent on the things that are specific to the target board. So, it consists of the code which is specific to that particular board, and it applies only to that board. The usual list includes Interrupt Request Numbers (IRQ), which are dependent on how the various devices are connected on the board. Also, some boards have an audio codec and you need to have a driver for that codec. Likewise, there would be switch interfaces, a matrix keypad, external `EEPROM`, and so on.

LSP placement

LSP is placed under a specific `<arch>` folder of the kernel's `arch` folder. For example, architecture-specific code for ARM resides in the `arch/arm` directory. This is about the code, but you also need the headers which are placed under `arch/arm/include/asm`. However, board-specific code is placed at `arch/arm/mach-<board_name>` and corresponding headers are placed at `arch/arm/mach-<soc architecture>/include`. For example, LSP for Beagle Board is placed at `arch/arm/mach-omap2/board-omap3beagle.c` and corresponding headers are placed at `arch/arm/mach-omap2/include/mach/`. This is shown in figure 4.

Machine ID

Every board in the kernel is identified by a machine ID. This helps the kernel maintainers to manage the boards based on ARM architecture in the source tree. This ID is passed to the kernel from the second stage bootloader such as u-boot. For the kernel to boot properly, there has to be a match between the kernel and the second stage boot loader. This information is available in `arch/arm/tools/mach-types` and is used to generate the file `linux/include/generated/`